README

boba user@1000 out-of-scope watchdog (BOB-116 / BOB-120 / BOB-123)

Revision: 2 **Last modified:** 2026-08-19T02:30:00Z **Class:** operator-run system-slice service — captures forensics on user@1000 SIGKILL **Anchors:** §11.4.4 four-layer, §11.4.108 runtime-signature, §11.4.115 RED-first, §11.4.6 anti-guessing, §11.4.10 credentials, §11.4.128 recording, §11.4.201 guard-real-condition, §11.4.161 rootless (no; system-slice by intent), §6.U (operator runs `su`, agent never invokes it).

Purpose

4 forced-logout incidents (BOB-116 / BOB-120 / BOB-123 + the 2026-07-07 one) share this mechanism: `user@1000.service` is SIGKILLed → cascade kill of every child process → GDM greeter respawns → operator perceives sudden logout.

Every in-scope monitor (user systemd timer, user cron, rootless podman container) DIES IN THE SAME CASCADE and cannot preserve forensics — it is architecturally incapable of observing the kill of the scope it runs in.

This watchdog is the ONLY mechanism that can attribute the SIGKILL because it lives in `system.slice` (a peer of `user.slice`), so it survives the kill of `user@1000.service` and captures the surrounding journal + cgroup + audit + process-tree state DURABLY.

What gets installed

Path	Purpose
`/usr/local/bin/boba-user1000-watchdog`	The monitor script (bash)
`/etc/systemd/system/boba-user1000-watchdog.service`	Systemd unit — `Slice=system.slice`, runs as root, `Restart=always`
`/var/log/boba-watchdog/YYYY-MM-DDTHH-MM-SSZ/`	Per-incident evidence dir, chmod 700

How it works

1. On boot, systemd starts the service in `system.slice` as root.
2. The script runs a pre-flight (§11.4.201): verifies uid=0 AND its own cgroup is (a) NOT under `/user.slice/` AND (b) POSITIVELY under `/system.slice/` (fail-closed rejection of cgroup-namespaced contexts where the negative check alone would pass while actually being inside user.slice). REFUSES to run in any other case.
3. NOTE: because the preflight requires the system.slice cgroup, a manual root-terminal debug invocation (`su -c '/usr/local/bin/boba-user1000-watchdog'` from a tty) will refuse — for that, use `su -c 'systemd-run --slice=system.slice --pty /usr/local/bin/boba-user1000-watchdog'` which drops the shell into system.slice for the duration (this host has no `sudo`, only `su`).
4. Enters a `journalctl -f -u user@1000.service` loop.
5. On matching the prefix `<TARGET_UNIT>: Main process exited` AND either `status=9` OR `code=killed`, checks the `CAPTURE_COOLDOWN_SEC` dedup gate; if outside cooldown, triggers `capture_forensics` immediately.
6. Captures: journal window (`PRE_KILL_WINDOW_SEC` before + `POST_KILL_WINDOW_SEC` after), cgroup state (memory + PSI + pids), full process tree, kernel audit log (if Path-1 audit rules installed), uptime + last, loginctl show-user.
7. Rotates evidence dirs to keep the last `RETAIN_LAST_N` (default 20).
8. Keeps tailing — captures every subsequent incident too.

Tunables (environment)

- `BOBA_WATCHDOG_TARGET` — target unit to monitor (default `user@1000.service`; set to a scratch unit for a §11.4.108 layer-4 install-time drill without touching production)
- `BOBA_WATCHDOG_ROOT` — evidence root (default `/var/log/boba-watchdog`)
- `BOBA_WATCHDOG_RETAIN` — retain last N incident dirs (default 20)
- `BOBA_WATCHDOG_PRE_SEC` — pre-kill journal window (default 60)
- `BOBA_WATCHDOG_POST_SEC` — post-kill journal window (default 15)
- `BOBA_WATCHDOG_COOLDOWN_SEC` — dedup window for cascade triggers (default 60; I1 fix — cascade lines within 1s must not evict real incident dirs)

Set via drop-in `/etc/systemd/system/boba-user1000-watchdog.service.d/env.conf` with `[Service]` + `Environment="BOBA_WATCHDOG_ROOT=/path"`.

`BOBA_WATCHDOG_ROOT` misconfiguration is destructive: rotation `rm -rf` targets ANY dir under the configured root. Never point it at `/`, `/home`, etc.

Install / uninstall

Per §6.U + §11.4.109, no agent tool invokes `su`/`sudo`. Install and uninstall scripts print `su -c '…'` commands you copy/paste at your terminal:

```
bash scripts/system-slice-watchdog/install.sh
        # prints commands to run
bash scripts/system-slice-watchdog/uninstall.sh
        # prints commands to run
```

Anti-bluff verification (§11.4.108 + §11.4.115)

Run the challenge WITHOUT root — verifies structural correctness:

```
bash challenges/scripts/
        user1000_watchdog_challenge.sh              # GREEN mode
POLARITY_MODE=red bash challenges/scripts/
        user1000_watchdog_challenge.sh  # golden-bad polarity
```

The challenge asserts:

1. All source files exist + parseable (`bash -n`, systemd unit shape)
2. All installer scripts are executable

3. **Load-bearing**: service declares `Slice=system.slice` (not `user.slice`)
4. Watchdog pre-flight refuses to run in user.slice
5. Evidence root is durable (`/var/log`, not tmpfs)
6. Journal trigger matches the exact BOB-116/BOB-120/BOB-123 signature
7. Rotation implemented (bounded disk)
8. Resource limits set (`MemoryMax`, `CPUQuota`, `TasksMax`)

After an incident

The forensics are in `/var/log/boba-watchdog/<timestamp>/`:

```
state.log         — uptime, last, loginctl, meminfo, PSI, cgroup
snapshot
journal-pre.log   — journal for PRE_KILL_WINDOW_SEC before kill
journal-post.log  — journal captured POST_KILL_WINDOW_SEC after
kill
proctree.log      — full ps auxf + top-20-by-RSS
audit.log        — kernel audit logs (requires Path-1 sudo audit
rules)
```

Compose with Path 1 (kernel audit rules)

For **initiator attribution** (PID + UID + cmdline of the SIGKILL sender), run the audit-rule install from `docs/incidents/2026-08-19-sudo-audit-rules-for-operator.md` BEFORE the next incident. The watchdog captures the audit log entries; without the rules, `audit.log` will say "(ausearch failed — audit rules may not be installed)".

Honest boundaries (§11.4.6)

- The watchdog DIAGNOSES; it does NOT prevent the kill.
- **M3 fix**: "survives the kill" is DESIGN INTENT proven at §11.4.108 layer-1 (source declares system.slice) + layer-2 (post-install `systemctl show boba-user1000-watchdog.service -p Slice` returns `system.slice`). Layers 3-4 (survives an actual kill event; captures durable evidence during real incident) are **PENDING** operator install + next incident. A cross-referable positive fact: GDM and auditd both survived all 4 forced-logout incidents on this host and both live in system.slice — this is external evidence of the category-survival, not proof of THIS unit's per-incident behavior.
- The watchdog CANNOT observe events pre-kill that happen outside `user@1000.service`'s journal — e.g. a mutter/gnome-shell segfault

would be in the user's journal which may already be flushed by the time the watchdog captures.
- The watchdog runs as root (needed to survive user@1000 kill + read all journals); it is bounded by `MemoryMax=200M` and `CPUQuota=15%` per §12.6.
- **I3 fix — credential leak honesty (§11.4.10)**: evidence dirs contain `ps auxf` and `ps -eo cmd` output. Those capture **full process cmdlines**, which on this host CAN contain credentials (curl -u, env-prefixed invocations, some tracker plugins). Mitigations: (a) evidence dirs are `root:root 0700`, (b) the watchdog itself logs ONLY operational lines to journal (trigger lines, filenames, preflight status) — never evidence contents, and no secret is passed through `log()`, (c) never commit raw captures unredacted per §11.4.128(4). If you export a capture for sharing, redact cmdline fields first.
- **M4 fix — KillUserProcesses claim was speculative**: prior text said "systemd's baked-in default is yes". Distro-patched behavior varies; the authoritative source is `busctl get-property org.freedesktop.login1 /org/freedesktop/login1 org.freedesktop.login1.Manager KillUserProcesses` at runtime — check it, don't assume.
- **Restart-gap-of-blindness**: `Restart=always RestartSec=5s` means there is a ~5 second window after any watchdog exit during which no capture will happen. If an incident hits inside that window, evidence will be sparse.